

الملف التقني لمنظومة أهلا للتواصل

هذا الملف يشرح "أهلا" من زاوية **الهندسة البرمجية والبنية التحتية** كما هي في الحزمة الأساسية:
ahla_v9_0_supersuite_20251109_201443 + حزم IaC و Go-Live المرفقة في المستودع.

يركّز الوصف على: - مكونات المنظومة (Front / Back / Realtime / AI / Omni / Storage). - طريقة ترابط الخدمات مع بعضها. - بنية التشغيل (Docker Compose محليًا، و Terraform/EKS إنتاجيًا). - طبقة المراقبة والأمن والامتثال. - تقييم جاهزية المنظومة وما تبقى من فجوات تقنية.

1. نظرة تقنية عليا (High-Level Architecture)

المنظومة مبنية كـ **مجموعة خدمات مصغرة (Microservices)** + واجهات ويب مستقلة، تُدار خلف **Nginx Reverse Proxy**، مع طبقة مراقبة كاملة (Prometheus + Grafana) وطبقة ذكاء (Emotion Engine).

1.1. الواجهات (Front-end Apps)

مجلد: apps/

- chat-web → واجهة محادثات.
- meet-web → واجهة الاجتماعات المرئية/الصوتية.
- drive-web → واجهة الملفات.
- business-web → واجهة المهام/الـ Business.
- mail-web → واجهة البريد.
- omni-web → واجهة موحّدة (Hub) تربط كل ما سبق.

جميعها مبنية بـ: **React 18 - Next.js 14 (App Router)** - تشغيل كحاويات مستقلة خلف Nginx على المسارات: /chat, /meet, /drive, /biz, /mail, /omni

في النسخة الحالية، الواجهات عبارة عن **قوالب جاهزة (Shells)** توضح الهيكل والتنقل، وقابلة للتوسع لاحقًا بتصاميم وهوية بصرية كاملة.

1.2. الخدمات الخلفية (Back-end APIs)

مجلد: services/

خدمات Python (FastAPI): chat-api → إدارة رسائل الغرف النصية (تخزين في Redis + API بسيط للسجل والاقتراحات). meet-api → جدولة الاجتماعات وتخزين بيانات أولية عن الاجتماعات. drive-api → استقبال الملفات ورفعها إلى مجلد تخزين /data (موصول على drive_data Volume). business-api → CRUD بسيط للمهام (Task Model) في الذاكرة. mail-api → نموذج أولي لصندوق بريد (رسائل ثابتة مبدئيًا). emotion-engine → محرك ذكاء بسيط لتحليل النص (الانطباع + النبذة) باللغة العربية.

القاسم المشترك: - جميعها مبنية على **FastAPI** . - جميعها مهيأة للتصدير إلى **Prometheus** عبر **starlette_exporter** على مسار **/metrics** . - كل خدمة تعمل في حاوية مستقلة ويمكن توسيعها وربطها بقاعدة بيانات حقيقية (PostgreSQL، إلخ) لاحقًا.

1.3. خدمات ال (WebSocket / WebRTC) Realtime

خدمات Node.js: **chat-ws** → خادم WebSocket لمسار **/ws** : - يفتح قناة WebSocket لكل عميل. - يتتبع عدد الاتصالات النشطة ويصدر مقياسًا (**ahla_ws_connections**) عبر مكتبة **prom-client** . - يميز الرسائل بين المستخدمين في القناة نفسها (Broadcast مبسط). - **meet-ws** → خادم WebSocket لمسار **/signal** : - يدير غرف الاجتماعات WebRTC. - يحتفظ بخريطة **rooms** (كل غرفة → قائمة WebSocket Clients). - يعيد بث رسائل ال **Signaling** (offers / answers / ICE candidates) بين الأطراف.

هذه الطبقة تمثل **قلب الزمن الحقيقي** في أهلا، ومهيأة للدمج مع خوادم STUN/TURN وخدمات CoTURN الموجودة في حزم ال IaC.

1.4. Omni Gateway ال

خدمة **omni-gateway** (**http-proxy-middleware** + Node.js): - توفر **طبقة Proxy موحدة** لنداءات **/api/*** :
/api/chat → **chat-api** - **/api/meet** → **meet-api** - **/api/drive** → **drive-api** - **/api/biz** → **business-api** - **/api/mail** → **mail-api** - تضيف نقطة دخول واحدة يمكن لاحقًا حمايتها بـ: بوابة API، توثيق JWT/SSO، سياسات معدل (WAFg Rate Limiting).

1.5. طبقة ال Reverse Proxy

داخل مجلد **infra/nginx/** : - ملف **nginx.conf** يوزع الحركة على: - الواجهات **/chat**، **/meet**، ...
 الخدمات الخلفية **/api/chat**، **/api/meet**، ... - Endpoints الزمن الحقيقي **/ws** و **/signal** مع تهيئة: - **Upgrade** و **Connection: Upgrade** لضمان WebSocket.

2. البنية التحتية والتشغيل (Runtime & Infra)

2.1. تشغيل محلي (Docker Compose)

المجلد: **infra/**

- ملف **docker-compose.yml** :
- حاوية **reverse-proxy** (Nginx).
- حاويات الواجهات الستة.
- حاويات الخدمات الخلفية (Chat/Meet/Drive/Business/Mail/Omni/Emotion Engine).
- حاويات دعم:
 - **redis** لتخزين رسائل المحادثة اللحظية.
 - **meilisearch** (مذكور في README) للفهرسة والبحث مستقبلاً.
 - **prometheus** + **grafana** للمراقبة.
 - **drive_data** Volume لتخزين ملفات أهلا درايف.

2.2. التشغيل الإنتاجي (AWS / Kubernetes)

المستودع يحتوي حزم جاهزة لـ البنية التحتية ككود (IaC):

1. `ahla_go_live_aws_v1_20251109_210505/terraform`
 2. وحدات (Modules) لبناء:
 - عنقود EKS.
 - RDS/Networking/NLB/ALB.
 - ربط Route53 Certificatesg.
 3. سكربتات نشر الحاويات على العنقود.
 4. `ahla_iac_v4_3_patch_keda_autoscale/`
 5. تهيئة **KEDA** كـ Autoscaler قائم على Prometheus.
 6. `ScaledObject` لخدّم CoTURN في namespace `ahla-system` ، يرفع ويخفض عدد الـ Pods حسب حركة الـ TURN.
 7. حزم أخرى (`ahla_ci_https_turn_pack_v1` , `ahla_aws_blocks_v1_1` , ...)
 8. تغطي:
 - شهادات HTTPS (ACM).
 - ExternalDNS لربط النطاقات.
 - إعدادات CoTURN كخدّم STUN/TURN.
- النتيجة: المنظومة قابلة للتشغيل إما **محليًا بالكامل** عبر Docker Compose، أو **إنتاجيًا على AWS EKS** عبر Helm Terraform، مع قابلية توسّع أفقية حقيقية.

3. البيانات والتخزين (Data & Storage)

في النسخة الحالية:

- **المحادثات (Chat):**
- تخزين آخر 500 رسالة لكل غرفة في Redis (`room:{room}:messages`).
- لا يوجد بعد **تخزين طويل الأمد** في قاعدة بيانات (يمكن ربط PostgreSQL لاحقًا).
- **الاجتماعات (Meet):**
- مخزّنة في الذاكرة (In-Memory Dict) داخل `meet-api` كمثال أولي.
- **الملفات (Drive):**
- استخدام مسار `/data` (Environment `DRIVE_STORE`) مع Volume `drive_data`.

- رفع الملف عبر UploadFile في FastAPI، وتخزينه باسم الملف الأصلي.
- المهام (Business):
- تخزين في الذاكرة (Dict) داخل business-api مع توليد UUID لكل مهمة.
- البريد (Mail):
- قائمة ثابتة (Mock) في MAIL، جاهزة لأن تُستبدل لاحقًا بمصدر فعلي (IMAP/SMTP أو خدمة بريد داخلية).
- من منظور هندسي: البنية الحالية تمثل طبقة نموذج أولي (MVP Data Plane)، تحتاج في مرحلة الإنتاج إلى: ربط جدي بـ PostgreSQL أو ما يمثله. - تصميم مخطط بيانات (Schema) لكل مجال (/Chat /Meet/Drive/Tasks/Mail). - إعداد نسخ احتياطي (Backup) واسترجاع.

4. المراقبة والرصد (Observability)

4.1 Metrics

- كل خدمات FastAPI مهيأة بـ PrometheusMiddleware + مسار /metrics.
- خدمات Node.js (chat-ws, meet-ws) تستخدم prom-client وتعرّف: ahla_ws_connections
- ahla_meet_ws لعدد الاتصالات في خدمة الإشارات.

4.2 Prometheus + Grafana

- infra/prometheus/prometheus.yml يحدد أهداف الـ Scrape.
- infra/grafana/ يحتوي على:
- Datasources جاهزة لـ Prometheus.
- Dashboards مبدئية لمراقبة:
 - عدد الاتصالات.
 - حركة HTTP.
 - صحة الخدمات.
- هذا يمثل أساسًا ممتازًا لبناء لوحة تشغيل تنفيذية (Executive Dashboard) لأهلا في البيئات الإنتاجية.

5. الأمن والامتثال (Security & Compliance)

في هذه المرحلة، المستودع يوفّر تهئية أولية لعناصر الأمن والامتثال، وليس منظومة أمن مكتملة.

5.1 قنوات الاتصال

- Nginx هو نقطة الدخول؛ يفترض في الإنتاج تشغيله خلف:

- HTTPS كامل (ACM / Cert-Manager).
- WAF في طبقة أعلى (AWS WAF أو ما يعادلها).

5.2. تنظيم البيانات وحقوق الأفراد

- مجلد `pdp1/` يحتوي على: `PRIVACY_POLICY_TEMPLATE.md` → قالب سياسة الخصوصية باللغة العربية. -
- `DSR_PROCEDURE.md` → إجراءات التعامل مع طلبات أصحاب البيانات (DSR).
- الهدف من هذه الملفات: - مواءمة المنظومة مع متطلبات **نظام حماية البيانات الشخصية السعودي (PDPL)**. -
- تعريف مسار عمل واضح لطلبات الاطلاع/التصحيح/المحو.

5.3. ما هو غير مفعل بعد (Gaps)

- لا توجد طبقة مصادقة/تحويل (AuthN/AuthZ) فعالة في هذه الحزمة (لا JWT/SSO/Keycloak بعد).
- لا يوجد تشفير End-to-End حقيقي في طبقة الرسائل (ال WebSocket حالياً يمرر البيانات كما هي).
- لا توجد سياسات Rate Limiting/WAF مضبوطة داخل الكود؛ تعتمد على الطبقات الخارجية.
- هذه الفجوات متوقعة في حزمة MVP، لكنها نقاط واضحة لخطط الإصدار القادمة.

6. تقييم الجاهزية التقنية (READY / NEAR-READY / NOT-READY)

6.1. جاهز (READY)

- **هيكل الخدمات** (APIs + Web + Realtime) متكامل ويعمل عبر Docker Compose.
- **المراقبة** جاهزة (Prometheus + Grafana مع Metrics من كل الخدمات).
- **منظومة الملفات (Drive)** تعمل فعلياً وتخزن الملفات في Volume مستقل.
- **Emotion Engine** جاهز للاستدعاء من الواجهات لتحليل النصوص.

6.2. شبه جاهز (NEAR-READY)

- **الانتقال إلى EKS/إنتاج:**
- بنية Terraform وKEDA وCoTURN جاهزة، لكنها تحتاج:
 - تعبئة القيم الفعلية (حساب AWS, VPC/Subnets, Domains, Certificates).
 - ربط الـ Pipelines (CI/CD) لتحديث الصور تلقائياً.
- **التكامل مع خوادم STUN/TURN:**
- حزم CoTURN/KEDA جاهزة.
- يلزم ضبط عناوين STUN/TURN في الـ WebRTC Clients واختبارها على شبكة حقيقية.

6.3. غير جاهز بعد (NOT-READY)

- **الأمن من طرف إلى طرف (E2EE) في المحادثات والاجتماعات:**
- لا يوجد حتى الآن تنفيذ فعلي لبروتوكولات مثل Signal/X3DH داخل الكود.

- قاعدة البيانات المركزية:
- الاعتماد الحالي على Redis والذاكرة/Volume للملفات؛ نحتاج طبقة بيانات مؤسسية.
- هوية موحدة (SSO) وربط مع مزود الهوية الوطنيين/المؤسسيين.

7. خارطة الطريق التقنية (Technical Roadmap - مختصرة)

1. إدخال طبقة Auth/SSO:
2. Keycloak أو ما يماثله، مع OIDC.
3. أدوار وصلاحيات (RBAC) لكل تطبيق (Chat/Meet/Drive/Mail/Biz).
4. تبني قاعدة بيانات مؤسسية:
5. PostgreSQL + مخطط بيانات موحد.
6. ترحيل البيانات من Redis/In-Memory إلى جداول حقيقية.
7. تفعيل WebRTC كامل + STUN/TURN إنتاجيًا:
8. ربط `meet-web` بالـ Signaling + STUN/TURN.
9. إضافة دعم الشاشة المشتركة والتسجيل (عند الحاجة وبالتوافق مع الخصوصية).
10. طبقة أمن متقدمة:
11. تطبيق تشفير End-to-End في المحادثات.
12. تفعيل WAF + Rate Limiting + Audit Logs.
13. تحسين تجربة الواجهات:
14. تصميم واجهات كاملة (RTL/عربية أولاً) لكل من Chat/Meet/Drive/Mail/Omni.

8. خلاصة تقنية تنفيذية

- من زاوية هندسية، "أهلاً" اليوم عبارة عن **SuperSuite** جاهزة للتشغيل محليًا بكامل الخدمات الأساسية (Chat/Meet/Drive/Mail/Business/Omni + Realtime + Emotion + Observability).
- المستودع يحتوي بالفعل على:
- بنية أساسية لـ Docker Compose (تجارب/تطوير).
- بنية Terraform/EKS + KEDA/CoTURN (للإنتاج السحابي).
- قوالب PDPL/DSR للامتثال القانوني.

على هذه القاعدة، يمكن الانتقال من نموذج MVP إلى **منصة تواصل وإنتاجية سيادية مؤسسية** بمجرد استكمال: -
طبقة الهوية والأمن. - طبقة البيانات المؤسسية. - وربط WebRTC/STUN/TURN في بيئة حقيقية.